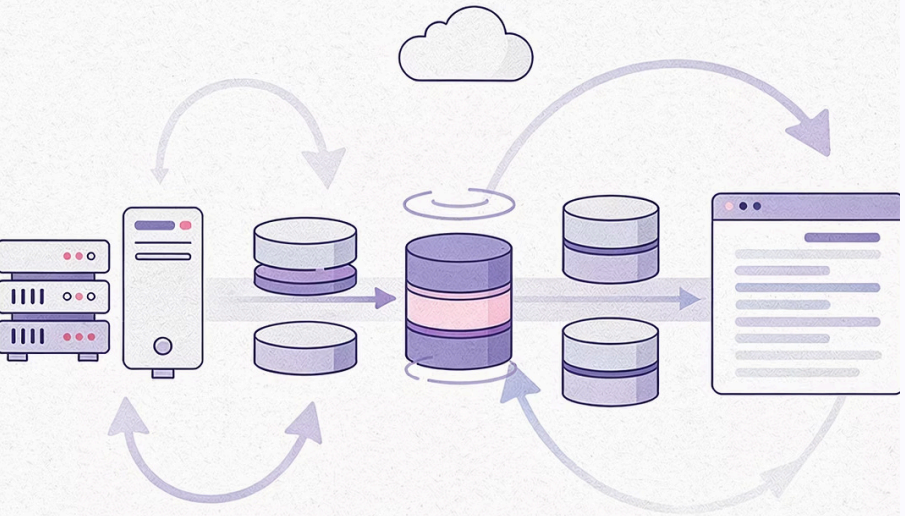
DATABRICKS & SPARK

PRACTICAL GUIDE

Data Transformations in Databricks

Learn how to choose efficient Spark transformations and build clear, scalable, production-friendly pipelines. Designed for data engineers and analytics engineers who already know the basics.



CHAPTER 1 FOUNDATIONS

What is a Data Pipeline?

A data pipeline is a **sequence of transformations** that moves data from one state to another. In Databricks, that usually means reading raw data, applying business and technical rules, and writing trusted outputs for downstream use.

Ingest

Read data from files, streams, APIs, or operational systems.

Transform

Clean, standardise, join, derive, and aggregate data into useful structures.

Serve

Publish tables that are reliable for analytics, reporting, or machine learning.

The core idea is simple: every transformation should move data closer to a defined business outcome while preserving performance and maintainability.

Transformation Types

In Spark, a useful mental model is to separate transformations into **narrow** and **wide**. This distinction matters because it strongly affects shuffle behaviour, execution cost, and runtime predictability.

Narrow

Each output partition depends on a single input partition. These operations are usually cheaper because data can stay local.

- select
- filter
- withColumn

Wide

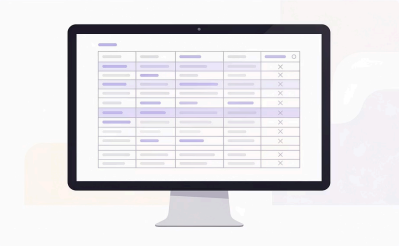
Output partitions depend on multiple input partitions. Spark must redistribute data across the cluster, which introduces a shuffle.

- join
- groupBy / aggregate
- distinct / repartition

When performance troubleshooting starts, first ask: **where are the wide transformations?** That question usually points to the expensive parts of the pipeline.

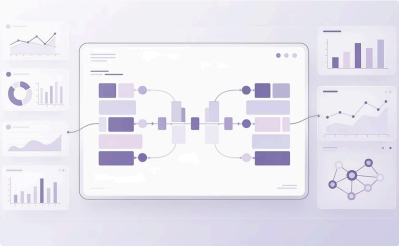
CORE OPERATIONS

Common Transformations



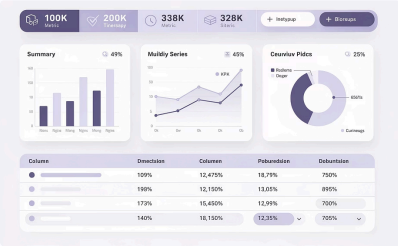
Filter

Reduce the dataset early by removing records you do not need. Early filtering lowers I/O, memory pressure, and downstream compute.



Join

Combine datasets using shared keys. Powerful, but often expensive, especially when both sides are large and require a shuffle.

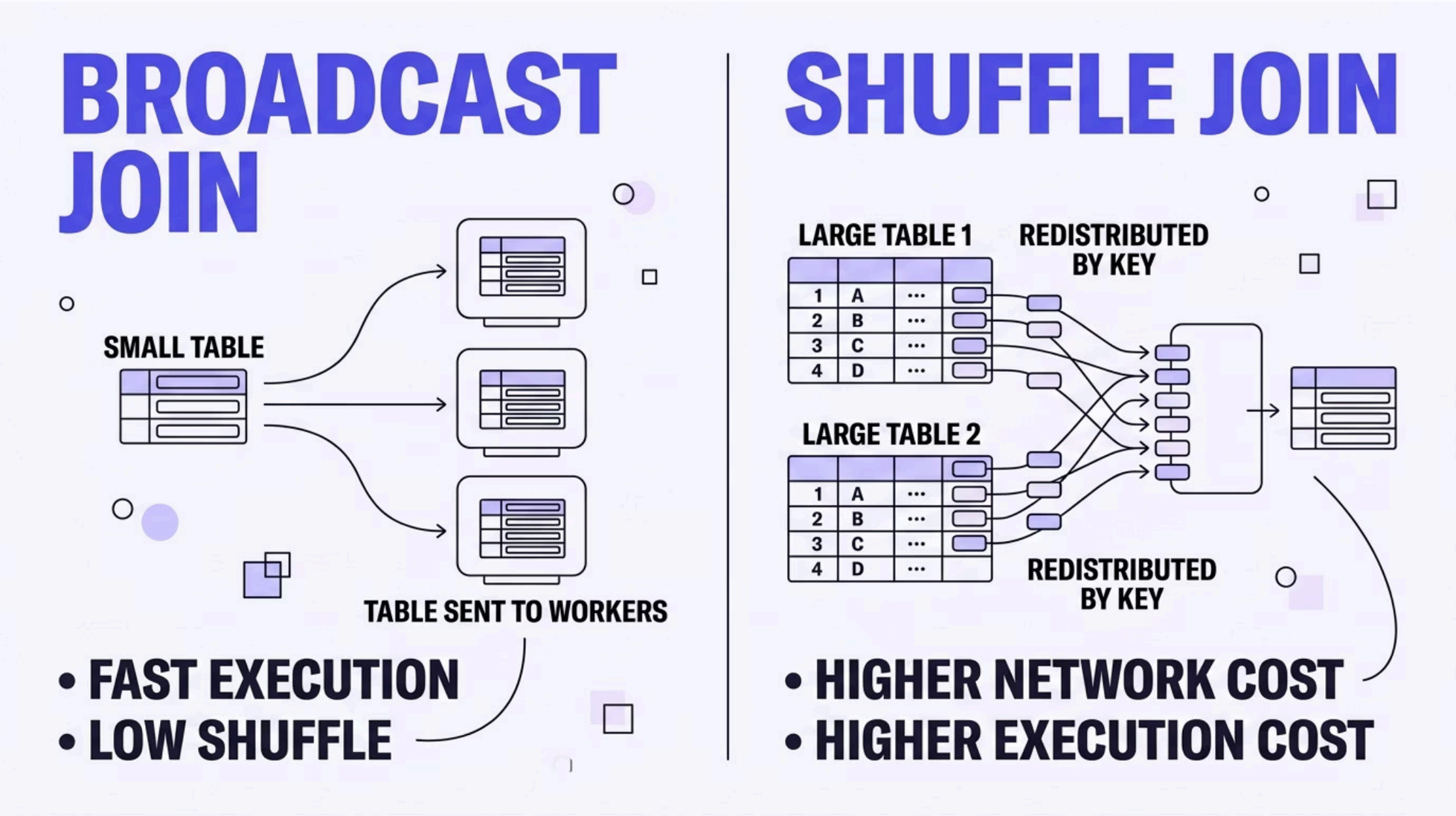


Aggregate

Summarise detailed records into metrics such as counts, sums, averages, and grouped business measures.

These three operations appear in nearly every real pipeline. The engineering challenge is not merely using them, but ordering them sensibly and controlling their cost.

Join Strategies



Use a **broadcast join** when one side is small enough to distribute to executors efficiently. Use a **shuffle join** when both sides are too large to broadcast. In practice, good join strategy depends on table size, key distribution, skew, and whether filtering can reduce either side before the join.

> Rule of thumb: shrink data before joining wherever possible. A smaller input often matters more than a clever optimisation later.

SCHEMA WORK

Handling Columns

Column-level transformations are common and usually inexpensive, but they are essential for readability and schema discipline. Well-managed columns make pipelines easier to test, review, and evolve.

Add

Use `withColumn` or select expressions to derive new business fields, flags, and standardised values.

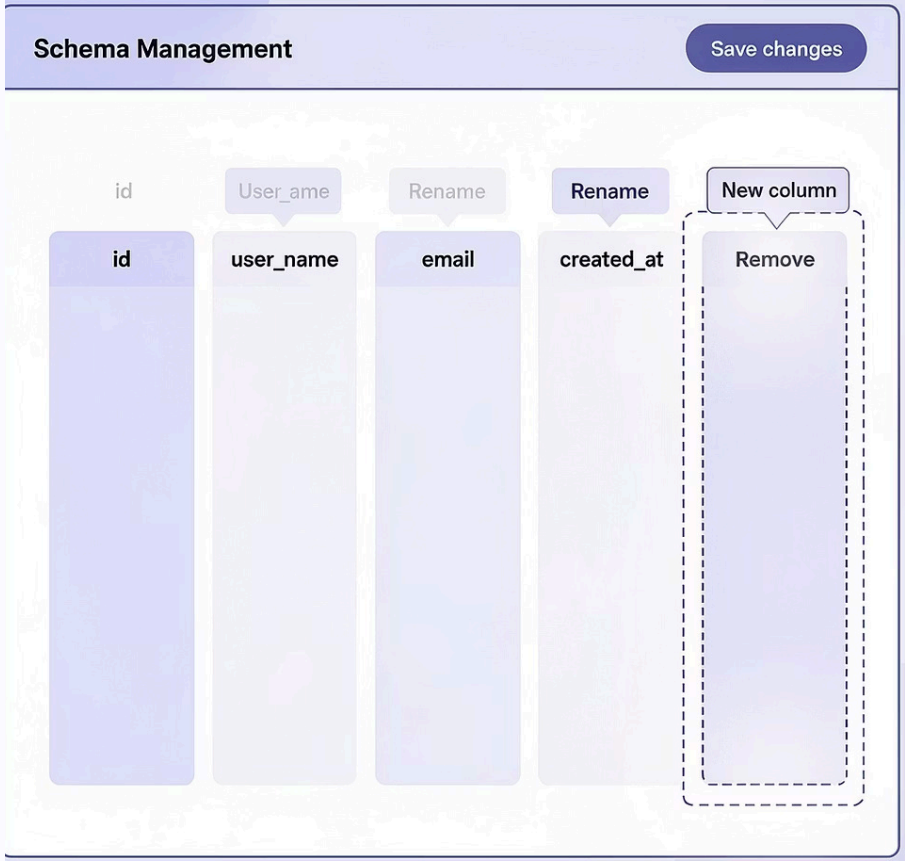
Rename

Apply consistent naming conventions so downstream users can understand the dataset without reverse engineering intent.

Drop

Remove temporary, duplicate, or irrelevant fields to keep outputs lean and unambiguous.

A strong pattern is to keep derivations explicit and finish with a deliberate `select` that defines the final contract of the table.



CRITICAL WARNING PERFORMANCE RISK

UDFs: Avoid If Possible

User-defined functions can be useful, but they should be a **last resort**. In many cases they reduce Spark's ability to optimise execution, making jobs slower and harder to reason about.

Why risky

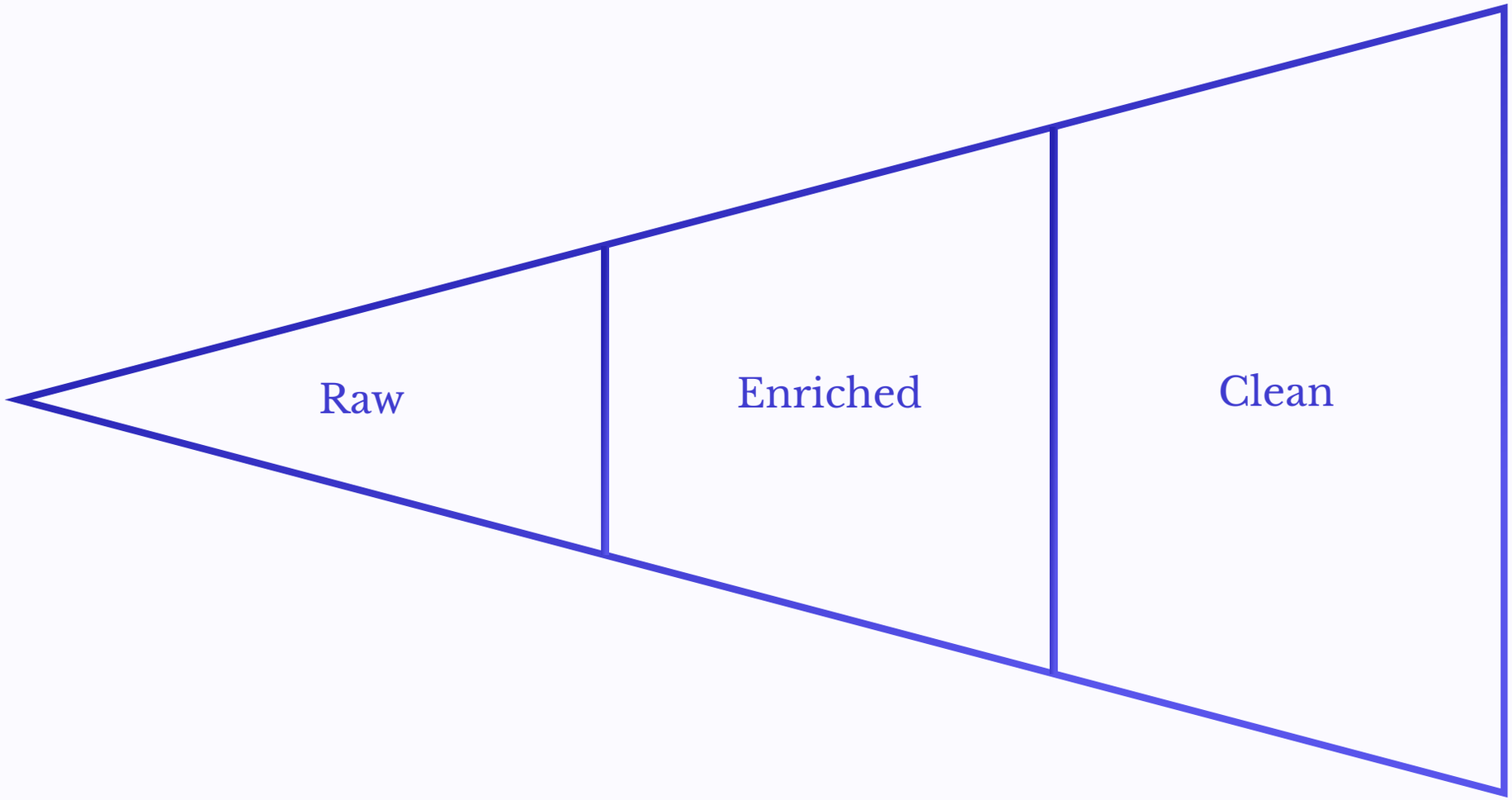
UDFs can limit Catalyst optimisation, add serialisation overhead, and make code less transparent to other engineers.

Preferred choice

Use built-in Spark SQL and DataFrame functions first. They are faster, better understood by the engine, and easier to maintain.

If you believe a UDF is necessary, pause and check whether the logic can be expressed with native functions, `when`, string/date helpers, arrays, maps, or SQL expressions.

Pipeline Pattern



This pattern gives structure to transformation design. **Raw** preserves source fidelity, **Clean** resolves data quality and schema consistency, and **Enriched** adds business context such as dimensions, derived fields, and reusable metrics. Separating these stages improves traceability, testing, and reuse across teams.

WHAT TO AVOID

Common Anti-Patterns

Too many shuffles

Repeated wide operations without reduction or planning can dominate runtime and cluster cost.

Unnecessary UDFs

Custom logic used where built-in Spark functions would be faster, clearer, and easier to optimise.

No partitioning

Poor data layout makes reads slower and limits efficient pruning during downstream workloads.

Most performance problems are not caused by one dramatic mistake, but by several small inefficiencies stacked together. Good pipelines stay intentional about shuffles, functions, and storage layout from the start.

NEXT STEP | HANDS-ON LAB

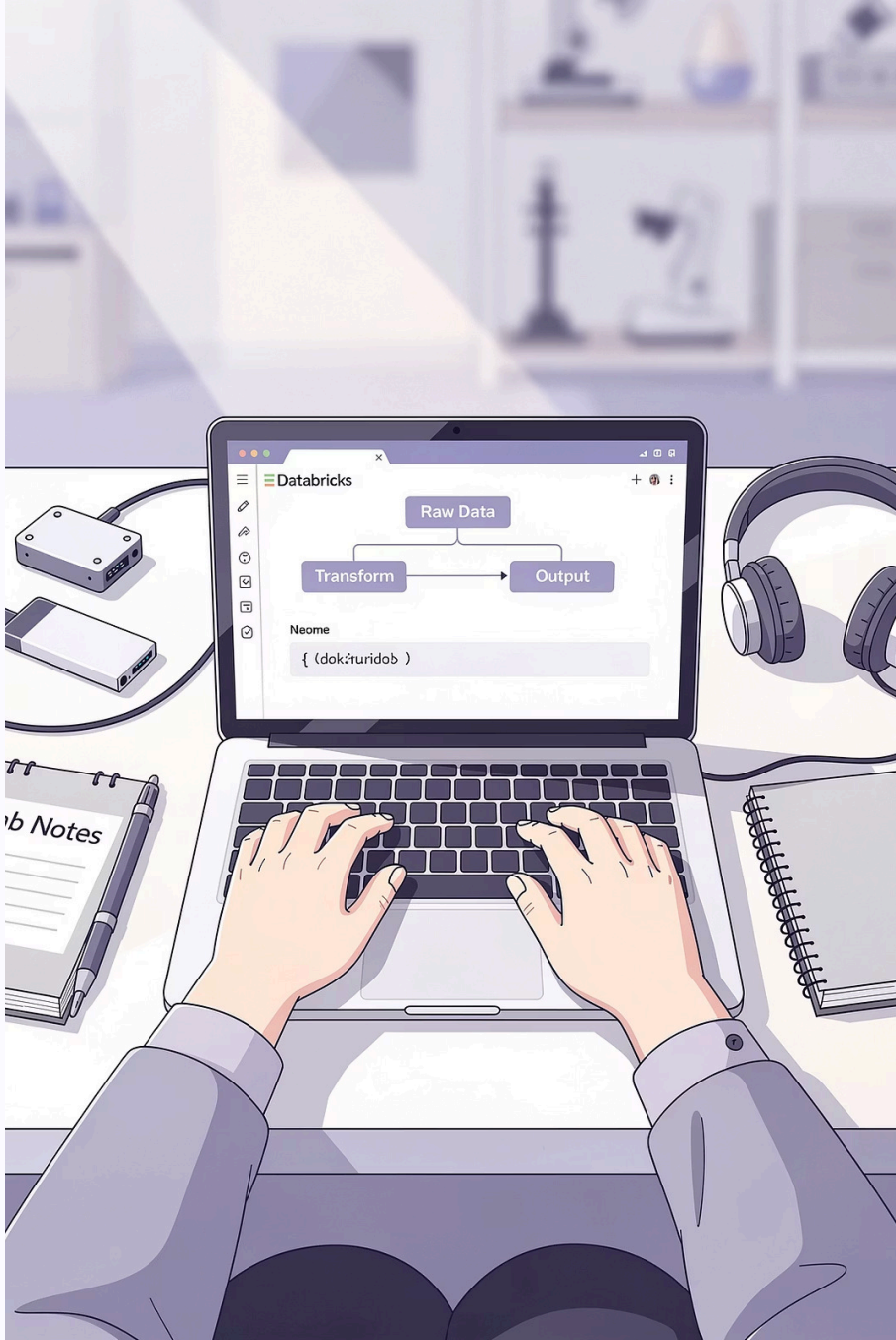
Now move from concepts to implementation. In the lab, you will create a simple Databricks pipeline that reads raw data, applies cleaning rules, enriches it with additional logic, and produces a final table with a clear transformation path.

→ Start with a small dataset

Keep the scope narrow so you can focus on transformation design and execution behaviour.

→ Use built-in functions

Prefer native expressions for filtering, column handling, joins, and aggregations.

→ Inspect the plan

Look for shuffles, verify join choices, and connect code structure to Spark execution.